

Claude Code 가이드 슬라이드 — 기술 문서 (v1.1)

작성일: 2026-06-27 대상 파일: `claude-code-guide.html` (+ 범용 템플릿 `slide-template.html`)

0. 이 문서의 목적

- 1. **학습용** — `claude-code-guide.html` 에 쓰인 기술스택과 패턴을 보고 공부하기 위함
 - 2. **점검용** — 전체 구조가 어떻게 짜여 있는지 확인하기 위함
 - 3. **재사용용** — 같은 패턴을 다른 발표자료(`slide-template.html`)로 일반화한 근거를 남김
- 이후 코드가 추가로 바뀌면, 맨 아래 "7. 변경 히스토리"에 버전을 추가해 누적 관리한다.

1. 기술 스택 개요

영역	선택	이유
마크업	순수 HTML5 (단일 파일)	빌드 도구·서버 없이 브라우저에서 바로 열림. 발표 직전 인터넷이 끊겨도 동작
스타일	순수 CSS3 (프레임워크 없음)	Tailwind/Bootstrap 의존성 없이 16:9 비율과 애니메이션을 세밀하게 직접 제어
스크립트	순수 JavaScript (바닐라, ES6)	슬라이드 전환·진행률·페이지번호 자동계산만 필요해 프레임워크가 과함
폰트	시스템 폰트 (Apple SD Gothic Neo / Malgun Gothic / Noto Sans KR)	웹폰트 다운로드 없이 오프라인에서도 즉시 렌더링

영역	선택	이유
그래픽	인라인 SVG (화살표 도형)	외부 이미지 파일 없이 코드로 직접 그려, 색을 인라인 그라디언트로 그대로 통일
레이아웃 단위	<code>vw</code> (뷰포트 너비 %)	<code>.stage</code> 를 16:9로 고정해두고 그 안의 모든 글자·여백을 <code>vw</code> 로 잡아, 화면 크기가 달라져도 비율이 깨지지 않음
인쇄/배포	<code>@media print</code> + Chrome <code>--print-to-pdf</code>	같은 HTML 파일이 화면 발표용과 PDF 배포용을 동시에 처리

"프레임워크 없는 단일 HTML 파일" 구조를 쓴 이유 — 이 발표자료는 "만들고 바로 쓰는" 용도라, 휴대성·즉시 실행성이 장기 유지보수성보다 중요했다. 파일 하나만 옮기면 어떤 컴퓨터에서도 더블클릭으로 똑같이 열린다.

2. 폴더 구조

```
class/
├─ claude-code-guide.html      ← 실제 발표 슬라이드 (24장, 단일 파일)
├─ claude-code-guide.pdf      ← 위 파일을 그대로 인쇄해 만든 배포용 PDF
├─ slide-template.html        ← claude-code-guide.html에서 내용만 비운 범용 템플릿 (8장)
├─ slide-template-guide.md/.pdf ← 템플릿 사용법 (클래스 목록, 애니메이션 규칙)
├─ tech-guide.md/.pdf         ← 지금 이 문서
└─ Sample_PORTFOLIO_TEMPLATE_GUIDE.md ← 포트폴리오 실습용 별도 가이드 (실습 1 슬라이드에서 참조)
```

3. HTML 문서 구조 한눈에 보기

```
<head>
  <style> ... 전체 CSS (디자인 + 애니메이션 + 인쇄 규칙) ... </style>
</head>
<body>
```

```

<div id="deck"><div class="stage">      ← 16:9 비율 고정 무대
  <div class="slide active">...</div>   ← SLIDE 00 표지 (그룹 없음)
  <div class="slide" data-group="main">...</div> ← SLIDE 01~10 본편 (10/10까지)
  <div class="slide">...</div>          ← 실습 구분 장 (그룹 없음)
  <div class="slide" data-group="practice">...</div> ← 실습 1~2 (포트폴리오 / 프레젠테이션)
  <div class="slide">...</div>          ← Appendix 구분 장 (그룹 없음)
  <div class="slide" data-group="appendix">...</div> ← APPENDIX A1~A7 부록
  <div class="slide">...</div>          ← 마무리 인사 (그룹 없음)
</div></div>
<div id="progress"></div>              ← 상단 진행률 바
<div id="nav">...</div>                ← 우측 상단 이전/다음 버튼
<script> ... 내비게이션 + 진행률 + 그룹별 페이지번호 자동계산 ... </script>
</body>

```

24개 `<div class="slide">` 가 `.stage` 안에 겹쳐서(`position: absolute`) 쌓여 있고, 한 번에 하나만 `.active` 클래스로 보인다 — 스크롤형이 아니라 "카드 갈아끼우기형(deck-style)" 구조다. 슬라이드 순서는 HTML에 적힌 순서 그대로다.

본편(main)·실습(practice)·부록(appendix)은 `data-group` 속성으로 구분된 서로 독립적인 그룹이다(각각 10개·2개·8개). 표지·구분장·마무리처럼 `data-group` 이 없는 슬라이드는 페이지 번호가 전체 절대 위치(`n/24`)로, 그룹이 있는 슬라이드는 자기 그룹 안에서만 번호가 매겨진다(본편은 10/10으로 끝나고, 부록은 다시 1/8부터 시작) — 자세한 구현은 5-3 참고.

4. CSS 핵심 패턴

4-1. 16:9 비율 고정 — `min()` 트릭

```
.stage { width: min(100vw, 177.78vh); height: min(100vh, 56.25vw); }
```

`177.78vh` 는 "뷰포트 높이의 177.78%"인데, 16:9 비율($16/9=1.7778$)이라 세로 기준으로 계산한 가로 길이다. `min(100vw, 177.78vh)` 는 "화면이 가로로 넓으면 높이에 맞추고, 세로로 길면 폭에 맞춘다"는 뜻 — 어떤 창 크기에서도 16:9가 절대 깨지지 않고 가운데 정렬된다 (영화관 스크린이 검은 띠로 letterbox 되는 것과 같은 원리).

4-2. vw 단위로 통일한 이유

Edu-report/briefing.html 은 clamp(rem, vw, rem) 을 썼지만, 이 파일은 .stage 자체가 이미 16:9로 폭이 고정되어 있어 vw 하나만 써도 "화면이 커지든 작아지든 같은 비율"이 보장된다. 다만 이 때문에 실제 인쇄용 페이지 비율이 16:9와 다르면(A4 등) 글자 크기 비율이 달라질 수 있다 — 그래서 @page 크기를 16:9에 맞춰 강제로 지정해줬다 (4-4 참고).

4-3. 슬라이드 구조 3단 — .si → .sb → .tip

```
.si { flex:1; display:flex; flex-direction:column; padding:4.5% 7.5% 3.5% 7.5%; }
.sb { flex:1; display:flex; flex-direction:column; justify-content:center; }
.tip { flex-shrink:0; margin-top:2.5%; }
```

.si (슬라이드 안쪽 전체) 안에 헤더(.sn / .h2 / .hsub) → .sb (본문, 남은 공간을 전부 차지하며 항상 가운데 정렬) → .tip (맨 아래 핵심 한 줄, .sb 의 형제 요소) 순으로 쌓는 구조를 모든 슬라이드에 동일하게 강제했다. 이 패턴을 안 지키면(.tip 을 .sb 안에 넣는 등) 슬라이드마다 하단 여백이 들쭉날쭉해지는 문제가 생긴다 — 실제로 이 프로젝트 초반에 겪은 문제였다.

4-4. 인쇄 전용 규칙 — @media print

```
@media print {
  .slide { width:100%; height:auto; aspect-ratio:16/9; page-break-after:always;
  @page { size: 13.333in 7.5in; margin:0; }
}
```

aspect-ratio:16/9 로 화면 비율을 강제하고, @page 용지 크기도 16:9 비율(13.333in × 7.5in, PowerPoint 16:9 표준과 동일)로 직접 지정해 인쇄 시 위아래 빈 여백이 생기지 않게 했다. -webkit-print-color-adjust:exact 는 크롬이 인쇄 시 배경색을 생략하는 기본 동작을 막아, 어두운 배경 디자인이 인쇄에서도 그대로 나오게 한다.

4-5. 애니메이션 3종

```
/* 순차 등장 */
.reveal-row > * { opacity:0; }
.slide.active .reveal-row > *:nth-child(1) { animation-delay:.05s; }
```

```

/* ...n번째 자식마다 delay 증가... */

/* 화살표+다음박스 쌍으로 등장 */
.reveal-flow > *:nth-child(2), .reveal-flow > *:nth-child(3) { animation-delay: 1s; }

/* 강조 깜빡임 — 무한반복 금지, 5회만 */
.slide.active .emph-blink { animation: pulseEmph 1s ease-in-out 5; }

```

`nth-child` 선택자로 "몇 번째 자식인지"에 따라 `animation-delay` 를 다르게 줘서 순차 등장 효과를 만든다. `.reveal-flow` 는 화살표 도형과 그 다음 박스를 같은 delay 값으로 묶어, "화살표가 나오고 한참 뒤에 박스가 따라나오는" 어색함 없이 한 쌍으로 같이 등장하게 만든 게 핵심이다. `emph-blink / glow-link` 는 `animation` 뒤에 반복 횟수(5)를 명시해 무한 반복(`infinite`)을 쓰지 않는다 — 발표 중 청중의 시선을 영원히 빼앗지 않기 위함이다.

5. JavaScript 핵심 패턴

5-1. 슬라이드 전환 — 클래스 토글 방식

```

function go(n, dir) {
  const next = (n + total) % total;          // 음수/초과 인덱스를 안전하게 순환
  outSlide.classList.add(dir > 0 ? 'leave-l' : 'leave-r');
  inSlide.classList.add(dir > 0 ? 'enter-r' : 'enter-l');
  requestAnimationFrame(() => requestAnimationFrame(() => {
    inSlide.classList.add('active');
    restartAnim(inSlide);
  }));
}

```

`display:none` 대신 `opacity + transform` 클래스를 토글해 CSS `transition` 이 작동하게 만든다. `requestAnimationFrame` 을 두 번 중첩한 이유는, 브라우저가 `enter-r` 클래스(시작 상태)를 화면에 한 번 그린 다음에 `active` 클래스(끝 상태)로 바뀌어야 트랜지션이 발생하기 때문이다 — 한 프레임만 기다리면 두 클래스가 같은 프레임에 적용되어 트랜지션이 생략될 수 있다.

5-2. 애니메이션 재실행 — `restartAnim`

```
function restartAnim(slide) {
  slide.querySelectorAll('.reveal-row > *, .emph-blink, ...').forEach(el => {
    el.style.animation = 'none';
    void el.offsetHeight; // 강제 리플로우 - 브라우저가 'none' 상태를 실제로 적용하게 함
    el.style.animation = '';
  });
}
```

CSS 애니메이션은 같은 클래스를 다시 추가해도 "이미 끝난 상태"로 인식해 재생되지 않는다.

`animation:none` 을 줬다가 `el.offsetHeight` 를 읽어 강제로 리플로우(reflow)를 발생시킨 뒤 다시 비워 주면, 브라우저가 애니메이션을 처음부터 새로 재생한다 — 뒤로 갔다가 다시 그 슬라이드로 와도 카드가 다시 순서대로 나타나는 이유다.

5-3. 페이지 번호 자동 주입 — 그룹별 독립 카운터

```
// 본편(main)·실습(practice)·부록(appendix)은 그룹 안에서만 번호를 센다
['main', 'practice', 'appendix'].forEach(group => {
  const groupSlides = Array.from(slides).filter(s => s.dataset.group === group);
  groupSlides.forEach((slide, i) => {
    const label = (i + 1) + ' / ' + groupSlides.length;
    const sn = slide.querySelector('.sn');
    // 이미 자체 "1/2" 페이지 표시가 있는 슬라이드(지침 전문 등)는 중복 표시를 피해 건너뛴
    if (sn && !sn.textContent.includes('/')) sn.insertAdjacentHTML('beforeend',
  });
});
// 표지/구분장/마무리처럼 그룹이 없는 슬라이드는 전체 절대 위치를 보여줌
slides.forEach((slide, i) => {
  if (slide.dataset.group) return;
  const label = (i + 1) + ' / ' + total;
  const sn = slide.querySelector('.sn');
  if (sn) sn.insertAdjacentHTML('beforeend', ' <span class="pn">· ' + label + ' <
  else { /* 표지/마무리 슬라이드는 .cv-foot-r에 삽입 */ }
});
```

우측 상단 `#nav` 의 카운터는 **화면 전용**(JS로 매번 갱신)이라 인쇄하면 사라진다. 그래서 페이지 번호를 **콘텐츠 안** (`.sn`)에 직접 텍스트로 삽입해두면, 화면에서도 보이고 인쇄/PDF에서도 똑같이 남는다.

처음에는 전체를 한 줄로(`(i+1)+'/'+total`) 세는 단일 카운터였는데, 본편·실습·부록을 분리하면서 **data-group** 속성 + 그룹별 필터링으로 바꿨다. 그룹이 없는 슬라이드(표지/구분장/마무리)는 여전히 전체 절대 위치를 쓴다. 추가로, 환경설정 지침처럼 슬라이드 자체에 이미 `1/2` 같은 수동 페이지 표시가 있는 경우 자동 번호를 한 번 더 붙이면 `1/2 · 3/8` 처럼 분수가 두 번 겹쳐 보이는 문제가 있었다 —

`sn.textContent.includes('/')` 로 이미 표시가 있는 슬라이드는 자동 주입을 건너뛰게 해서 해결했다. 슬라이드를 추가/삭제해도 각 그룹의 길이가 자동 재계산되어 번호를 손으로 고칠 필요가 없다.

5-4. 터치 스와이프

```
document.addEventListener('touchend', e => {
  const dx = e.changedTouches[0].clientX - tx;
  if (Math.abs(dx) > 40) dx < 0 ? go(cur+1, 1) : go(cur-1, -1);
}, { passive: true });
```

touchstart 시점의 X좌표를 저장해두고 **touchend** 시점과 비교해, 40px 이상 좌/우로 스와이프하면 다음/이전 슬라이드로 넘어가게 한다. **passive:true** 는 브라우저에 "이 핸들러는 스크롤을 막지 않는다"고 미리 알려줘 스크롤 성능 경고를 없앤다.

6. SVG 화살표 기본 패턴

```
<svg viewBox="0 0 50 200" preserveAspectRatio="none">
  <defs><linearGradient id="g1" x1="0" y1="0" x2="1" y2="0">
    <stop offset="0%" stop-color="rgba(186,117,23,0.34)"/>
    <stop offset="100%" stop-color="rgba(186,117,23,0.056)"/>
  </linearGradient></defs>
  <polygon points="0,0 50,100 0,200" fill="url(#g1)"/>
</svg>
```

`viewBox="0 0 50 200"` 로 내부 좌표계를 정의하고, `polygon points` 로 삼각형(화살표 머리 모양)을 그린 뒤 `linearGradient` 로 왼쪽이 진하고 오른쪽이 투명해지는 그라디언트를 입힌다.

`preserveAspectRatio="none"` 은 부모 컨테이너 크기에 맞춰 화살표가 늘리거나 늘어나도 좌표 비율을 유지하지 않고 꽉 채우게 한다 — 각 슬라이드의 화살표 칸 너비가 달라도 같은 SVG 코드를 재사용할 수 있는 이유다.

7. 작업 시 유의사항 (이 프로젝트에서 실제로 겪은 문제들)

- `.tip` 을 `.sb` 형제로 두지 않으면 하단 정렬이 깨진다 — 반드시 `.si > .sb + .tip` 구조 유지
- `<table>` 은 브라우저가 자동으로 `<tbody>` 를 끼워넣어 `table.reveal-row > tr` 선택자가 작동하지 않는다 — `table.reveal-row tr` (직속이 아닌 후손 선택자)로 별도 지정 필요
- 표 첫 번째 열이 넓어지는 문제 — 그리드/테이블은 칸을 균등 분배하므로, 짧은 라벨 열은 `white-space:nowrap; width:1%;` 로 강제로 줄여야 함
- `emph-blink` 를 한 슬라이드에 여러 곳 쓰면 효과가 상쇄된다 — 슬라이드당 1곳으로 제한
- 인쇄 시 용지 크기를 사용자가 직접 A4 등으로 바꾸면 `@page` 커스텀 크기가 무시될 수 있다 — CSS만으로 100% 보장은 안 되므로, 배포 전 실제 인쇄 미리보기로 한 번 확인 필요
- 부록(Appendix)을 가리키는 텍스트(예: "Appendix A5 확인")는, 슬라이드를 삭제/재배치/재번호할 때 함께 깨지기 쉽다 — 이 프로젝트에서 어펜디스 순서를 4번 정도 바꾸는 동안 실제로 여러 번 끊긴 참조가 나왔다. 순서를 바꾼 뒤에는 `grep -n "Appendix A"` 로 전체 검색해 번호가 다 맞는지 확인하는 게 거의 필수 단계였다
- 그룹별(`data-group`) 페이지 번호와 슬라이드 자체의 수동 페이지 표시("1/2" 등)가 같은 슬라이드에 같이 있으면 분수가 겹쳐 보인다 — 자동 주입 로직에서 `.sn` 텍스트에 이미 `/` 가 있으면 건너뛰도록 예외 처리 필요 (5-3 참고)
- 타이틀(`.h2`) 폰트 크기를 슬라이드마다 다르게(예: 2.85vw~5vw) 쓰면 한눈에 봤을 때 위계가 흐트러진 것처럼 보인다 — 길어서 안 들어가는 제목은 폰트를 줄이기보다 텍스트를 줄여서 같은 크기를 유지하는 쪽이 전체 통일감에 더 유리했다
- 디자인 토큰(CSS 변수) 없이 색상을 인라인으로 직접 써서, 톤을 일괄로 바꾸려면 값을 일일이 찾아 바꿔야 한다 — 색상 종류가 더 늘어날 프로젝트라면 `:root` 변수로 옮기는 게 유지보수에 유리함 (현재는 슬라이드 24장 규모라 실익이 크지 않아 보류)

8. 변경 히스토리

v1.0 — 2026-06-27 (최초 작성)

- 단일 HTML 슬라이드 덱(19장) 완성, 16:9 고정 + vw 기반 반응형
- 순차 등장(`reveal-row`)·페어 등장(`reveal-flow`)·강조 깜빡임(`emph-blink`)·링크 강조(`glow-link`)·줄단위 타이프업(`tline`) 애니메이션 구축
- 인쇄/PDF 내보내기(`@media print` + 16:9 커스텀 용지) 및 화면·인쇄 공용 페이지 번호 자동 주입 시스템 구축

- 위 패턴을 일반화해 재사용 템플릿(`slide-template.html`) + 사용 가이드 분리

v1.1 — 2026-06-27 (구조 개편)

- 슬라이드 19장 → **24장**으로 확장: 실습 섹션(포트폴리오/프레젠테이션 만들기 2장) + GitHub·Vercel 단독 가이드 2장 신설
- 단일 전체 번호 시스템을 **data-group** 기반 그룹별 독립 카운터(본편 10/10, 실습 1/2, 부록 1/8)로 교체 — 5-3 참고
- 부록·실습 진입부에 "구분 장"(divider slide) 패턴 추가 — **.h1** 공용 클래스를 재사용해 표지와 톤을 맞춤
- 본편/부록 전체 타이틀(**.h2**) 폰트 크기를 5vw로 통일, 길어서 안 맞는 제목은 폰트 대신 텍스트를 줄여 한 줄 유지
- 모바일 뷰포트(390×844 기준 CDP 디바이스 에뮬레이션으로 검증) 및 보안 항목(외부 리소스·인라인 스크립트 개수·디버그 코드 잔존 여부) 최종 점검 완료

다음 업데이트부터는 위 형식으로 **v1.2** , **v1.3** ... 항목을 이 절 아래에 추가한다.